

Advanced Data Handling & Manipulation

Ensuring Resource Safety and Lifecycle and Management

MEMORY EFFICIENCY

Why Advanced Data Handling Matters

- Data volume changes the core cost model of simple Python operations.
- Materializing lists and tables multiplies memory overheads at scale.
- Lazy pipelines process items individually, keeping RAM bounds stable.
- Vectorization replaces expensive interpreter loops with optimized compiled binaries.
- Robust pipelines make datatypes, schemas, and memory footprint limits explicit.

```
memory_efficiency.py

def read_numbers(path):
    """Generator: reads numbers line by line without
    holding the whole file in memory."""
    with open(path, encoding="utf-8") as f:
        for line in f:
            if line.strip():
                yield int(line.strip())

def main():
    # Write a dummy numbers file
    with open("numbers.txt", "w") as f:
        f.write("5\n12\n3\n20\n8\n15\n")

    # Generator expression (lazy transformation) --
    # nothing runs yet, this just describes the pipeline
    pipeline = (
        value * 2
        for value in read_numbers("numbers.txt")
        if value > 10
    )

    # Values are computed on demand during iteration
    for value in pipeline:
        print(f"Processed: {value}")
    # ... (rest of code truncated for brevity)
```

WHAT THIS CODE DOES

- Streams numbers.txt one line at a time via a generator, never loading the whole file into a list.
- Filters (value > 10) and doubles each qualifying number lazily, inside a generator expression.
- Only the values actually iterated in the for loop are ever computed — here, 12, 20, and 15.

LAZY PIPELINES

1. Generators, Iterators and Lazy Evaluation

- The Iterator protocol uses `__iter__` and `__next__` for sequential item fetching.
- Generator functions yield values on demand rather than storing full series.
- Generator expressions establish deferred data calculations elegantly.
- Avoid premature sequence materialization (loading arrays into standard lists).
- Construct lazy pipelines to stream and transform heavy data sequences safely.

```
lazy_pipelines.py

def read_numbers(path):
    """Generator: reads numbers line by line without
    holding the whole file in memory."""
    with open(path, encoding="utf-8") as f:
        for line in f:
            if line.strip():
                yield int(line.strip())

def main():
    # Write a dummy numbers file
    with open("numbers.txt", "w") as f:
        f.write("5\n12\n3\n20\n8\n15\n")

    # Generator expression (lazy transformation) --
    # nothing runs yet, this just describes the pipeline
    pipeline = (
        value * 2
        for value in read_numbers("numbers.txt")
        if value > 10
    )

    # Values are computed on demand during iteration
    for value in pipeline:
        print(f"Processed: {value}")
    # ... (rest of code truncated for brevity)
```

WHAT THIS CODE DOES

- Same generator pipeline as the previous slide, shown again to introduce the underlying iterator mechanics.
- `read_numbers` implements the iterator protocol implicitly: `yield` makes it satisfy `__iter__` / `__next__` automatically.
- The pipeline itself is a second, chained lazy layer built directly on top of that generator.

ITERTOOLS

2. itertools for Memory Friendly Processing

- itertools provides lightweight, composable stream processing blocks.
- islice enables safe, bounded record consumption without array duplications.
- groupby groups sequential matching elements, but requires sorted inputs.
- accumulate calculates running totals or cumulative operations on active streams.
- Combine lazy transformers to maintain processing boundaries across scopes.

```
itertools_processing.py

from itertools import islice, groupby, accumulate

def main():
    # 1. Bounded consumption with islice
    stream = (x * x for x in range(100))
    first_five = list(islice(stream, 5)) # only 5 values ever computed
    print(f"First five squares: {first_five}")

    # 2. Grouping sequential records with groupby
    data = [
        {"category": "A", "val": 10},
        {"category": "A", "val": 20},
        {"category": "B", "val": 15},
        {"category": "B", "val": 30},
    ]
    # groupby expects sorted inputs by the key function
    data.sort(key=lambda x: x["category"])
    for key, group in groupby(data, key=lambda x: x["category"]):
        group_items = list(group)
        print(f"Category {key} has items: {group_items}")

    # 3. Running calculations with accumulate
    # ... (rest of code truncated for brevity)
```

WHAT THIS CODE DOES

- islice(stream, 5) pulls only the first 5 squares from an infinite-looking generator — range(100) is never fully consumed.
- Sorting by category first is required: groupby only groups items that are already adjacent to each other.
- Produces two groups here — Category A (10, 20) and Category B (15, 30) — each still lazy until list(group) is called.

DATA SCHEMAS

3. Structured Data and `post_init` Validation

- Dataclasses provide lightweight, structured container models for data rows.
- Specify explicit parameter annotations and fields to prevent variable naming conflicts.
- Implement `__post_init__` validation to intercept and validate records early.
- Default factories prevent shared mutable array allocations across instances.

```
data_schemas.py

from dataclasses import dataclass, field

@dataclass
class Customer:
    name: str
    age: int
    tags: list[str] = field(default_factory=list) # avoids one shared list

    def __post_init__(self):
        """Validates age and cleans string whitespace
        after instance creation."""
        self.name = self.name.strip() # normalise, don't just check
        if self.age < 0:
            raise ValueError("Age must be zero or positive.")
        if not self.name:
            raise ValueError("Customer name cannot be empty.")

def main():
    try:
        valid_user = Customer(" Ada Lovelace ", 36, ["pioneer"])
        print(f"Valid: {valid_user}")
        # This will raise a ValueError
    # ... (rest of code truncated for brevity)
```

WHAT THIS CODE DOES

- `default_factory=list` gives every `Customer` its own tags list; without it, all instances would share one mutable list.
- `__post_init__` both cleans the data (stripping whitespace from name) and validates it, before the object is considered created.
- Constructing `Customer(" Ada Lovelace ", 36, ["pioneer"])` succeeds and stores the trimmed name "Ada Lovelace".

SERIALIZATION

4. Portability and Safe Custom JSON Encoding

- JSON is standard and highly portable, but lacks native complex type support.
- Subclass JSONEncoder and override default() to parse custom model schemas.
- Restore objects using granular dict expansion to rebuild dataclass structures.
- Pickle is highly insecure and vulnerable to injection attacks from untrusted data.
- Prefer JSON for external exchanges, reserving pickle only for closed internal states.

```
serialization.py

import json
from dataclasses import dataclass, asdict
from datetime import datetime

@dataclass
class Event:
    name: str
    timestamp: datetime # json.dumps can't handle this type alone

class CustomEncoder(json.JSONEncoder):
    def default(self, obj):
        if isinstance(obj, datetime):
            return obj.isoformat() # convert to a JSON-safe string
        return super().default(obj) # keep normal behaviour otherwise

def main():
    event = Event("system_boot", datetime.now())
    # Serialize to JSON using the custom encoder
    payload = json.dumps(asdict(event), cls=CustomEncoder)
    print(f"JSON Payload: {payload}")
    # ... (rest of code truncated for brevity)
```

WHAT THIS CODE DOES

- asdict(event) turns the Event dataclass into a plain dict first; only its datetime field still needs special handling.
- default() is only called for types json doesn't already know how to encode — here, that's just the datetime.
- Produces a JSON string like {"name": "system_boot", "timestamp": "2026-01-01T00:00:00"}.

5. Chunked Processing of Bulk Text Files

- Loading complete database dumps or raw CSV files easily triggers system out of memory.
- DictReader yields rows iteratively as dictionary streams, protecting heap memory.
- Aggregate data points directly on the fly rather than keeping every row in memory.
- Perform chunked summaries of files to write incremental output streams.
- Keep CSV ingestion workloads strictly bounded and predictable under load.

```
stream_file_io.py

import csv

def generate_csv_rows(path):
    """Yields rows incrementally from a CSV file."""
    with open(path, newline="", encoding="utf-8") as f:
        yield from csv.DictReader(f) # delegate to DictReader's own iterator

def main():
    # Write a dummy sales CSV
    with open("sales.csv", "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow(["category", "amount"])
        writer.writerow(["electronics", "299.99"])
        writer.writerow(["books", "15.50"])
        writer.writerow(["electronics", "45.00"])
        writer.writerow(["books", "120.00"])

    totals = {}
    for row in generate_csv_rows("sales.csv"):
        category = row["category"]
        amount = float(row["amount"])
    # ... (rest of code truncated for brevity)
```

WHAT THIS CODE DOES

- `yield from csv.DictReader(f)` streams rows one at a time straight from disk, never holding the full CSV in memory.
- Each row is aggregated into totals as it arrives, so memory use depends on the number of categories, not the file size.
- With this data, the running totals end at `{'electronics': 344.99, 'books': 135.50}`.

6. Vectorization and Groupby Transformations

- Standard row loops like `iterrows` are slow bottlenecks inside pandas.
- Use vectorized column expressions to leverage underlying optimized C operations.
- Groupby combined with named aggregations summarizes data into compact tables.
- Use groupby transform to perform operations while retaining original row alignments.
- Validated joins and explicit index mergers make dataset integrations robust.

```
advanced_pandas.py

import pandas as pd

def main():
    df = pd.DataFrame({
        "category": ["A", "A", "B", "B", "C"],
        "amount": [10.0, 20.0, 15.0, 35.0, 50.0],
    })

    # 1. Groupby named aggregation -- collapses to one row per category
    summary = (
        df.groupby("category", as_index=False)
        .agg(
            total=("amount", "sum"),
            average=("amount", "mean")
        )
    )
    print("Grouped Summary:")
    print(summary)

    # 2. Groupby transform (retains index alignment for original dataframe)
    df["group_total"] = df.groupby("category")["amount"].transform("sum")
    df["share"] = df["amount"] / df["group_total"]
    # ... (rest of code truncated for brevity)
```

WHAT THIS CODE DOES

- `.agg()` collapses the DataFrame to one summary row per category (A, B, C), each with its own total and average.
- `.transform("sum")` instead keeps all 5 original rows, attaching each row's category total alongside it.
- `share` then divides each row's amount by its own category's total, giving each row's percentage contribution.

7. Narrow dtypes and Categorical Optimization

- Repetitive text labels inside pandas columns consume substantial memory blocks.
- Cast low-cardinality string columns to category dtypes to optimize data structures.
- Use narrow numeric representations (float32 or int16) instead of heavy 64-bit defaults.
- Define explicit dtypes inside read_csv to optimize initial memory allocations.
- Process datasets incrementally in chunks using chunksize parameters.

```
pandas_diagnostics.py

import pandas as pd

def main():
    # Write a dummy sales CSV
    df_dummy = pd.DataFrame({
        "category": ["electronics", "books", "electronics", "clothing"] * 100,
        "amount": [299.99, 15.50, 45.00, 120.00] * 100
    })
    df_dummy.to_csv("large_sales.csv", index=False)

    # Explicit narrow datatypes definition
    dtype_spec = {
        "category": "category", # cuts down repetitive string storage
        "amount": "float32"      # 32-bit floats instead of default float64
    }

    totals = {}
    # Load and process CSV in chunks -- never all 400 rows at once
    for chunk in pd.read_csv("large_sales.csv", dtype=dtype_spec, chunksize=100):
        # Category aggregation inside each chunk
        partial = chunk.groupby("category", observed=True)["amount"].sum()
    # ... (rest of code truncated for brevity)
```

WHAT THIS CODE DOES

- dtype_spec is applied as the CSV is read, so columns never pass through pandas' heavier int64/float64/object defaults at all.
- chunksize=100 processes the 400-row file in four 100-row pieces, each small enough to hold comfortably in memory.
- Each chunk's partial category totals would then need combining across chunks to get the true file-wide totals.

8. Memory Tracking and Diagnostic Audits

- tracemalloc measures active memory allocations and captures state differences.
- Measure peak memory rather than final footprint to evaluate data loader bottlenecks.
- Use `memory_usage` with `deep=True` to audit columns and locate overhead leaks.
- Evaluate views vs copies in dataframe allocations to prevent duplicate arrays.
- Always base performance optimizations on empirical diagnostic measurements.

```
diagnostic_profiling.py

import tracemalloc

def execute_heavy_allocation():
    # Materializes a large list in memory
    return [x * 2 for x in range(500_000)]

def main():
    # Start tracking memory allocations
    tracemalloc.start()
    # Initial snapshot
    initial_snapshot = tracemalloc.take_snapshot()

    # Perform allocation
    data = execute_heavy_allocation()

    # Get current and peak memory usage
    current, peak = tracemalloc.get_traced_memory()
    print(f"Current Memory Usage: {current / (1024 * 1024):.2f} MB")
    print(f"Peak Memory Allocation: {peak / (1024 * 1024):.2f} MB")
    # Take final snapshot and stop
    # ... (rest of code truncated for brevity)
```

WHAT THIS CODE DOES

- `tracemalloc.start()` begins recording every allocation from this point on, until `tracemalloc.stop()` is called.
- `current` is the memory still held right now; `peak` is the highest point reached during the whole run — they can differ if memory was freed.
- Building the 500,000-item list is the single largest allocation this snippet performs, and dominates both figures.

8 Activities to Consolidate Concepts

1

Lazy Data Loader Generator → Refactor list materializations into generators.

2

itertools Stream Transformations → Slicing and sequential grouping lazily.

3

Dataclass Orders Validation → Schema constraints with `__post_init__`.

4

Safe Custom JSON Serialization → Custom datetime encoders using `asdict`.

5

Incremental CSV Chunk Processing → `DictReader` row streams and summaries.

6

pandas Column Vectorization → C-optimized arrays instead of row iteration.

7

pandas Memory Footprint Optimization → Downcasting numeric scales and categories.

8

End-to-End Safe Processing Pipeline → Stream-validate-load ETL pipeline.